
Prompts vs Agents

Contents

Executive Summary	2
AI Builder Grounded Prompt	3
What it does well.....	3
Example	3
Self-Hosted Agent	4
Example	4
In a PracticePro365 Scenario	5
Grounded Prompt	5
Self-Hosted Agent	5
ALM Comparison	6
Grounded Prompt	6
Self-Hosted Agent	6
What I'd Recommend for PracticePro365	6
Use Grounded Prompts when:	6
Use an Agent when:.....	6
My architecture choice	7

If you're evaluating **AI Builder Grounded Prompts** versus a **self-hosted agent (Copilot Studio, custom Azure AI agent, or proprietary agent architecture)**, they solve different problems even though both can answer questions over business data.

Executive Summary

Capability	AI Builder Grounded Prompt	Self-Hosted Agent
Setup effort	Low-code, minutes to hours	Moderate to high
Data source	Dataverse tables	Virtually anything
Conversational memory	Limited	Full conversation state
Multi-step reasoning	Minimal	Strong
Actions/automation	Usually via Power Platform	Full orchestration
Cost	Lower	Higher
Governance	Built into Power Platform	Must be designed
Best for	Q&A, summaries, classifications	Business assistants, workflows, automation

AI Builder Grounded Prompts are essentially a **grounded AI capability embedded in Power Platform**. You provide instructions and connect Dataverse data, and the LLM answers using that business data rather than relying only on general knowledge.
[\[learn.microsoft.com\]](https://learn.microsoft.com), [\[github.com\]](https://github.com)

A self-hosted agent is a broader architecture that can reason, decide, invoke tools, execute actions, maintain context, and connect to multiple data sources.

AI Builder Grounded Prompt

The Microsoft Learn module and AB-410 lab demonstrate creating a prompt grounded directly against Dataverse tables. The example connects a Work Order table and allows service agents to ask natural language questions about customer history.

[\[learn.microsoft.com\]](https://learn.microsoft.com), [\[github.com\]](https://github.com)

What it does well

- ✓ Dataverse search
- ✓ Record summarization
- ✓ Natural-language Q&A
- ✓ Power Automate integration
- ✓ Power Apps integration
- ✓ Solution-aware deployment

The lab specifically shows:

- Creating a prompt
- Connecting it to a Dataverse table
- Selecting columns available to the AI
- Passing runtime variables
- Returning answers grounded in business records [\[github.com\]](https://github.com)

Example

User asks:

"Has Contoso Manufacturing had any critical incidents in the last 6 months?"

Grounded Prompt:

1. Searches selected Dataverse rows
2. Retrieves matching records
3. Generates a response from those records

No custom orchestration required. [\[github.com\]](https://github.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

Self-Hosted Agent

Think of a self-hosted agent as:

```
1 User
2 ↓
3 Agent
4 ↓
5 Tools
6 |─ Dataverse
7 |─ SharePoint
8 |─ SQL
9 |─ APIs
10 |─ Power Automate
11 |─ Azure Functions
```

The agent can:

- Search data
- Decide next steps
- Call APIs
- Update records
- Trigger workflows
- Maintain conversation context

This is much closer to a digital employee.

Example

User says:

"Show all critical work orders for Fabrikam, create a summary, email it to the service manager, and create a follow-up task."

A self-hosted agent could:

1. Query Dataverse
2. Generate the summary
3. Find the manager
4. Send email
5. Create task
6. Report completion

A grounded prompt alone cannot orchestrate that end-to-end workflow.

In a PracticePro365 Scenario

Suppose you have:

- | | |
|---|--------------|
| 1 | Patients |
| 2 | Cases |
| 3 | Appointments |
| 4 | Documents |
| 5 | Notes |

Grounded Prompt

Good for:

"Summarize all interactions for Patient X"

"Have we seen this issue before?"

"What was the previous treatment?"

The response is generated from Dataverse records. [\[github.com\]](https://github.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

Self-Hosted Agent

Good for:

"Review open appointments, identify gaps, email providers, and create follow-up activities."

That requires reasoning and actions, not just retrieval.

ALM Comparison

From a Power Platform governance perspective:

Grounded Prompt

Can be packaged inside solutions and moved between environments as a Power Platform component. Prompt builder artifacts are solution-aware and reusable across apps, flows, and copilots. [\[learn.microsoft.com\]](https://learn.microsoft.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

Self-Hosted Agent

Requires managing:

- Agent definitions
- Connectors
- API credentials
- Azure resources
- Deployment pipelines
- Monitoring

More flexibility, more complexity.

What I'd Recommend for PracticePro365

Given the types of questions you've been asking around model-driven apps and Dataverse:

Use Grounded Prompts when:

- Users need answers from Dataverse
- Users need record summaries
- Users need natural language search
- You want a low-code solution

Use an Agent when:

- The AI must take actions
- Multiple systems are involved
- Workflows span SharePoint, Teams, Dataverse, Outlook, and external apps
- Users expect an ongoing conversation and task execution

My architecture choice

For most PracticePro365 deployments:

1	Grounded Prompt
2	+
3	Power Automate

is usually sufficient and significantly easier to support.

Move to a full agent only when users start asking:

"Do something for me"

instead of

"Tell me something about the data."

That's the key dividing line between the two approaches.

Creating a Local Instance of AI Agents

Running AI agents locally offers significant advantages, such as enhanced **privacy**, **cost-effectiveness**, and **offline accessibility**. Below is an overview of how you can achieve this using tools like **LangChain**, **LangGraph**, **Ollama**, and open-source models like **Qwen 3** or **Mistral**.

Easiest Self-Hosting Agent to Set Up and Run

If you want the **simplest self-hosting agent** to get running quickly, **Ollama** is widely regarded as the easiest option for developers and even non-technical users who want to run local AI models.

Why Ollama is the easiest

- **One-command setup:** You can install it on Linux, macOS, or Windows with a single command, and it runs a local OpenAI-compatible API toolradar.com.
- **No deep server config:** It handles Docker, networking, and API exposure automatically.
- **Broad model support:** Pulls and runs open-weight LLMs (e.g., Llama 3, Mistral) with minimal configuration.
- **Developer-friendly:** Scriptable, reproducible, and works well in CI/CD pipelines toolradar.com.

- **Free to self-host:** You only pay for hardware and optional cloud tiers [toolradar.com](https://www.toolradar.com).

Alternative for GUI-driven ease

If you prefer a **graphical interface** and don't want to work in the terminal, **LM Studio** is a polished desktop app that makes self-hosting feel like using a regular app [toolradar.com](https://www.toolradar.com). It's especially good on Apple Silicon and offers a clean, battery-included experience.

When to choose OpenClaw

If you want to self-host **multiple apps** (e.g., Nextcloud, Jellyfin, Immich) without writing config files, **OpenClaw** is the easiest "app store"-style self-hosting platform [HostAdvice](https://www.hostadvice.com). It manages Docker images, networking, and updates for you, so you can install and run several services from one dashboard.

Recommendation

- **For pure AI/LLM self-hosting:** Start with **Ollama** — it's the fastest to get running and most flexible.
- **For a GUI-friendly experience:** Try **LM Studio**.
- **For multiple self-hosted apps:** Use **OpenClaw**.

All three are free to self-host, so you can try them without upfront cost, and you only pay for the hardware you own.

Key Steps to Build Local AI Agents

1. Set Up Your Environment

To begin, you need to prepare your local machine with the necessary tools and libraries. Install Python 3 and create a virtual environment to manage dependencies. Tools like **Ollama** simplify running large language models (LLMs) locally by handling model configuration and GPU utilization.

Example installation for Ollama:

```
# Install Ollama (Linux/macOS)
```

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
# Verify installation
```

```
ollama --version
```


2. Choose and Download a Model

Select an open-source LLM like **Qwen 3** or **Mistral** based on your hardware capabilities. For instance:

- **Qwen 3** offers dense and Mixture-of-Experts (MoE) models for various tasks.
- **Mistral** is another lightweight model optimized for local use.

Download the model using Ollama:

```
# Pull a model (e.g., Qwen 3 or Mistral)
```

```
ollama pull qwen3:8b
```

3. Define AI Agent Tools

AI agents can interact with external tools (e.g., APIs, databases, or custom functions). Define these tools in Python using libraries like **LangChain** or **LangGraph**. For example:

```
from langchain.agents import tool
```

```
@tool
```

```
def get_current_datetime(format: str = "%Y-%m-%d %H:%M:%S") -> str:
```

```
    """Returns the current date and time."""
```

```
    import datetime
```

```
    return datetime.datetime.now().strftime(format)
```

4. Set Up the AI Agent

Use frameworks like **LangChain** or **LangGraph** to create the agent. These frameworks allow you to define workflows, integrate tools, and manage the agent's reasoning process. For example:

```
from langchain_ollama import ChatOllama
```

```
from langchain.agents import create_tool_calling_agent
```

```
# Initialize the LLM
```

```
llm = ChatOllama(model="qwen3:8b", temperature=0)
```

```
# Create the agent
```

```
agent = create_tool_calling_agent(llm, tools=[get_current_datetime])
```

5. Run the Agent

Once the agent is set up, you can invoke it with user queries. The agent will use its reasoning capabilities and tools to generate responses.

```
response = agent.invoke({"input": "What is the current date?"})
```

```
print(response['output'])
```

6. Advanced Features

For more complex tasks, you can:

- Use **Retrieval-Augmented Generation (RAG)** to query private documents by integrating vector databases like **ChromaDB**.
- Build multi-agent workflows with **LangGraph**, which supports loops, branching, and persistence.

Benefits of Running AI Agents Locally

- **Privacy:** Data never leaves your machine, ensuring confidentiality.
- **Cost Savings:** Avoid API fees by using open-source models.
- **Customization:** Fine-tune models and workflows to meet specific needs.
- **Offline Functionality:** Operate without relying on internet connectivity.

Considerations

- **Hardware Requirements:** Running large models locally requires sufficient RAM and VRAM. Smaller models like Qwen 3 (4B) or quantized versions are suitable for resource-constrained systems.
- **Model Configuration:** Ensure compatibility between tools, models, and frameworks to avoid errors during setup.

By leveraging tools like **Ollama**, **LangChain**, and **LangGraph**, you can create powerful, autonomous AI agents tailored to your specific requirements, all while maintaining full control over your data and infrastructure.